

Class 7: Machine Learning 1

Kai Christman (A17522126)

Table of contents

Background	1
K-means clustering	1
Hierarchical Clustering	8
Principal Component Analysis (PCA)	10
Analysis of UK Food Data	11
Tidy Data	11
Previewing the First 6 rows of the data	12
Exploratory Analysis	13
PCA	15

Background

Today we will explore some core machine learning methods that are very popular in bioinformatics. These include **clustering** and **dimensionality reduction**. The goal of **clustering** is to find groups within the data.

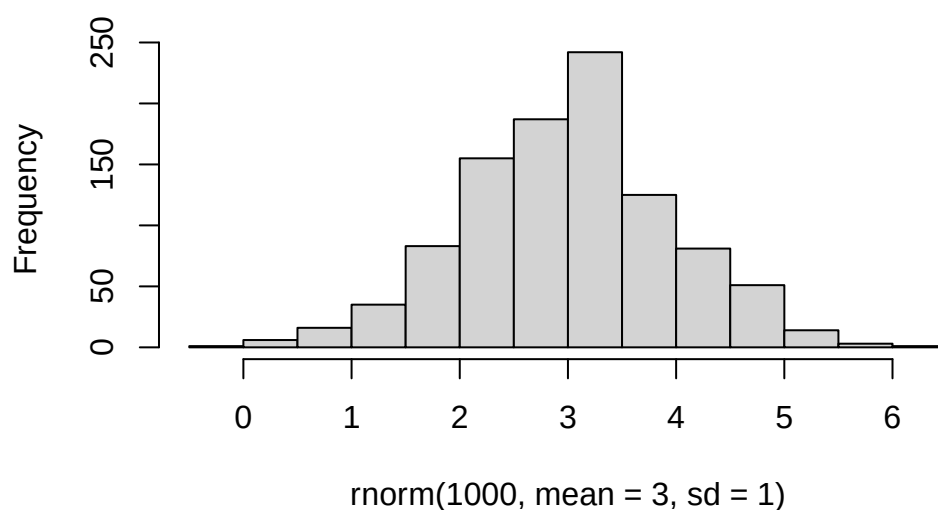
K-means clustering

The main function in “base” R for K-means clustering is called `kmeans()`.

Before we dive too deeply, let’s make up some “simple” data that we can cluster and know if we are getting a good answer. To assist with this process we can use `rnorm()` function:

```
hist( rnorm(1000, mean = 3, sd = 1) )
```

Histogram of rnorm(1000, mean = 3, sd = 1)



```
x <- c( rnorm(30, -3), rnorm(30, 3) )
```

```
x
```

```
[1] -1.3880362 -2.2728831 -2.6986360 -2.8073884 -1.4026785 -4.0339261
[7] -3.8170342 -3.0511672 -4.5152197 -3.2332118 -3.3303479 -2.7541403
[13] -0.8562322 -2.8971328 -2.9047369 -2.3840932 -2.0880091 -2.1597765
[19] -3.7746376 -2.3005426 -2.9930529 -3.3467745 -5.1165227 -4.5822822
[25] -3.4226066 -3.2971590 -3.0027846 -3.5448981 -4.2893715 -3.5043499
[31]  2.3247198  3.4657242  2.6256424  1.3880055  4.3514539  2.0678640
[37]  2.2671502  0.5553538  3.6037266  5.0606509  2.1230022  2.1174259
[43]  2.6699149  4.0574983  2.4044071  2.7719345  3.0399037  3.0278372
[49]  3.1140065  3.0827133  2.1706216  4.2827975  2.9710293  3.9893939
[55]  2.3583693  3.4012272  3.0111528  3.6244851  1.8253451  3.8469829
```

The reverse function reverses the order of the vector, positives are now first and negatives are second.

```
z <- cbind(x = x, y = rev(x))
```

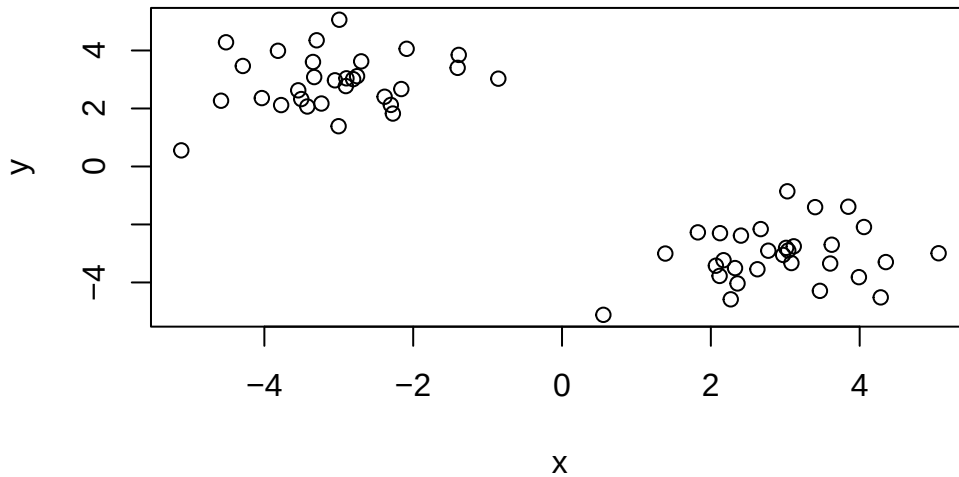
```
z
```

```
      x      y
[1,] -1.3880362  3.8469829
```

[2,]	-2.2728831	1.8253451
[3,]	-2.6986360	3.6244851
[4,]	-2.8073884	3.0111528
[5,]	-1.4026785	3.4012272
[6,]	-4.0339261	2.3583693
[7,]	-3.8170342	3.9893939
[8,]	-3.0511672	2.9710293
[9,]	-4.5152197	4.2827975
[10,]	-3.2332118	2.1706216
[11,]	-3.3303479	3.0827133
[12,]	-2.7541403	3.1140065
[13,]	-0.8562322	3.0278372
[14,]	-2.8971328	3.0399037
[15,]	-2.9047369	2.7719345
[16,]	-2.3840932	2.4044071
[17,]	-2.0880091	4.0574983
[18,]	-2.1597765	2.6699149
[19,]	-3.7746376	2.1174259
[20,]	-2.3005426	2.1230022
[21,]	-2.9930529	5.0606509
[22,]	-3.3467745	3.6037266
[23,]	-5.1165227	0.5553538
[24,]	-4.5822822	2.2671502
[25,]	-3.4226066	2.0678640
[26,]	-3.2971590	4.3514539
[27,]	-3.0027846	1.3880055
[28,]	-3.5448981	2.6256424
[29,]	-4.2893715	3.4657242
[30,]	-3.5043499	2.3247198
[31,]	2.3247198	-3.5043499
[32,]	3.4657242	-4.2893715
[33,]	2.6256424	-3.5448981
[34,]	1.3880055	-3.0027846
[35,]	4.3514539	-3.2971590
[36,]	2.0678640	-3.4226066
[37,]	2.2671502	-4.5822822
[38,]	0.5553538	-5.1165227
[39,]	3.6037266	-3.3467745
[40,]	5.0606509	-2.9930529
[41,]	2.1230022	-2.3005426
[42,]	2.1174259	-3.7746376
[43,]	2.6699149	-2.1597765
[44,]	4.0574983	-2.0880091

```
[45,] 2.4044071 -2.3840932
[46,] 2.7719345 -2.9047369
[47,] 3.0399037 -2.8971328
[48,] 3.0278372 -0.8562322
[49,] 3.1140065 -2.7541403
[50,] 3.0827133 -3.3303479
[51,] 2.1706216 -3.2332118
[52,] 4.2827975 -4.5152197
[53,] 2.9710293 -3.0511672
[54,] 3.9893939 -3.8170342
[55,] 2.3583693 -4.0339261
[56,] 3.4012272 -1.4026785
[57,] 3.0111528 -2.8073884
[58,] 3.6244851 -2.6986360
[59,] 1.8253451 -2.2728831
[60,] 3.8469829 -1.3880362
```

```
plot(z)
```



Now we can run `kmeans()` on this input `z` and see what the results look like!

```
km <- kmeans(x = z, centers = 2)
```

```
km
```

K-means clustering with 2 clusters of sizes 30, 30

Cluster means:

```
      x      y
1  2.920011 -3.058988
2 -3.058988  2.920011
```

Clustering vector:

```
[1] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1
[39] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

Within cluster sum of squares by cluster:

```
[1] 53.7836 53.7836
(between_SS / total_SS =  90.9 %)
```

Available components:

```
[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"       "
```

```
attributes(km)
```

\$names

```
[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"       "
```

\$class

```
[1] "kmeans"
```

Q. How many points are in each cluster?

```
km$size
```

```
[1] 30 30
```

Q. What component of your result object details cluster assignment/membership?

```
km$cluster
```

```
[1] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1  
[39] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

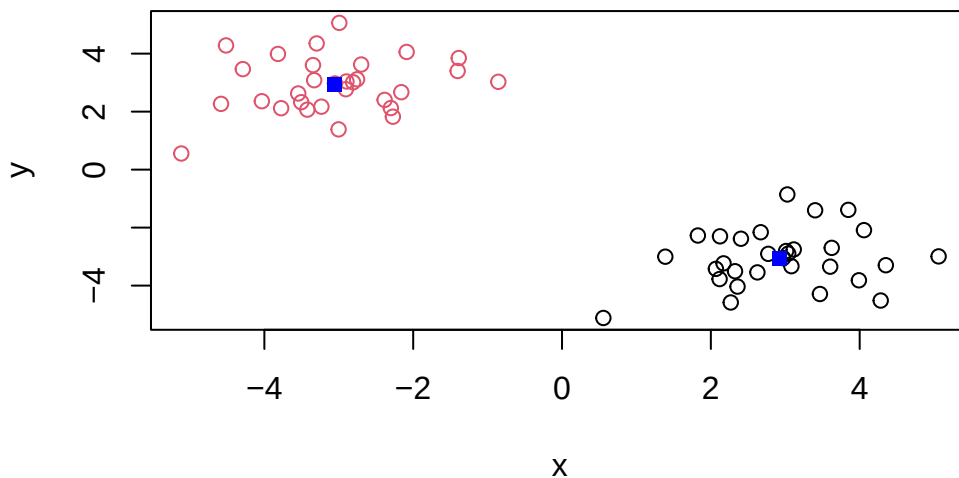
Q. What component of your result obbject details cluster center?

```
km$centers
```

```
      x      y  
1  2.920011 -3.058988  
2 -3.058988  2.920011
```

Q. Plot z colored by the kmeans cluster assignmnet and add cluster centers as blue points:

```
plot(z, col = km$cluster)  
points(km$centers, co = "blue", pch=15)
```



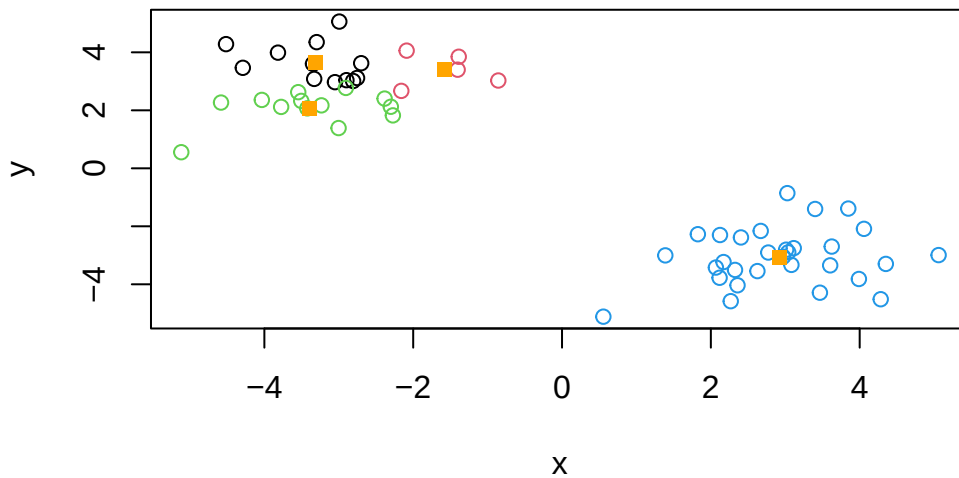
Q. Run a K-means clustering and plot the results asking for 4 clusters (K=4):

```
km4 <- kmeans(x = z, centers = 4)
```

```
km4$cluster
```

```
[1] 2 3 1 1 2 3 1 1 1 3 1 1 2 1 3 3 2 2 3 3 1 1 3 3 3 1 3 3 1 3 4 4 4 4 4 4 4  
[39] 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4
```

```
plot(z, col = km4$cluster)  
points(km4$centers, col = "orange", pch=15)
```

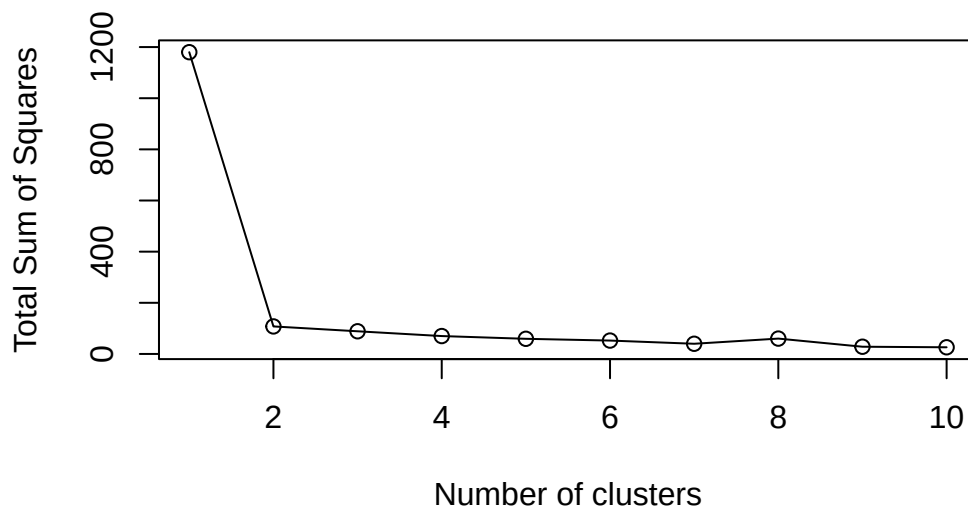


N.B. You need to tell K-means the number of clusters (i.e. set of `centers = 2`)!!

One approach is to try different values for `centers` and pick the best...

```
ans <- NULL  
  
for(i in 1:10) {  
  km <- kmeans(z, centers = i)  
  ans <- c(ans, km$tot.withinss)  
}
```

```
plot(ans, typ="o",
      xlab = "Number of clusters",
      ylab = "Total Sum of Squares")
```



Hierarchical Clustering

The main function in “base” R for Hierarchical Clustering is called `hclust()`. This will work in bottom up fashion. Starts with each data point being in its own cluster, grouping some of them together until everything is in one cluster at the end.

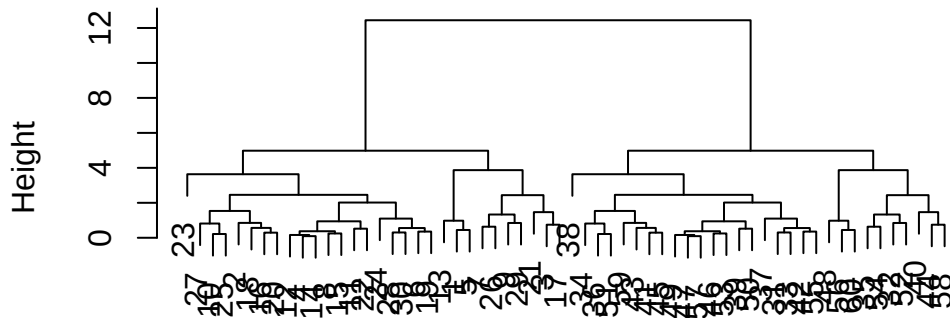
This function does not take your “raw” data for clustering. You must first build a “distance matrix” from your data and pass this as input to `hclust()`

```
d <- dist(z)
hc <- hclust(d)
```

This is a bespoke `plot()` method for `hclust()` result objects.

```
plot(hc)
```

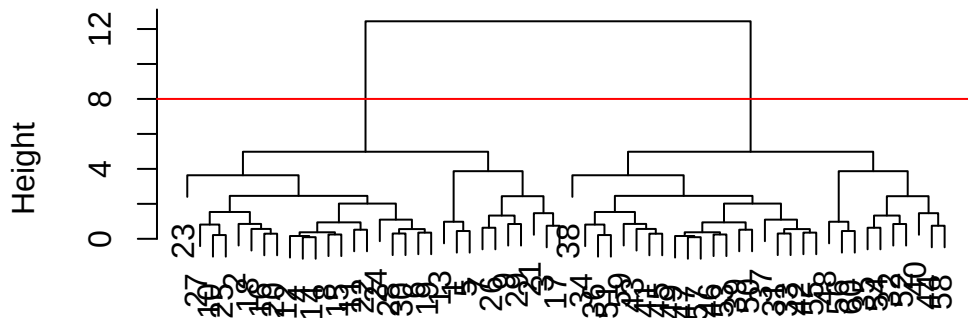

Cluster Dendrogram



d
hclust (*, "complete")

```
plot(hc)
abline(h=8, col = "red")
```

Cluster Dendrogram



d
hclust (*, "complete")

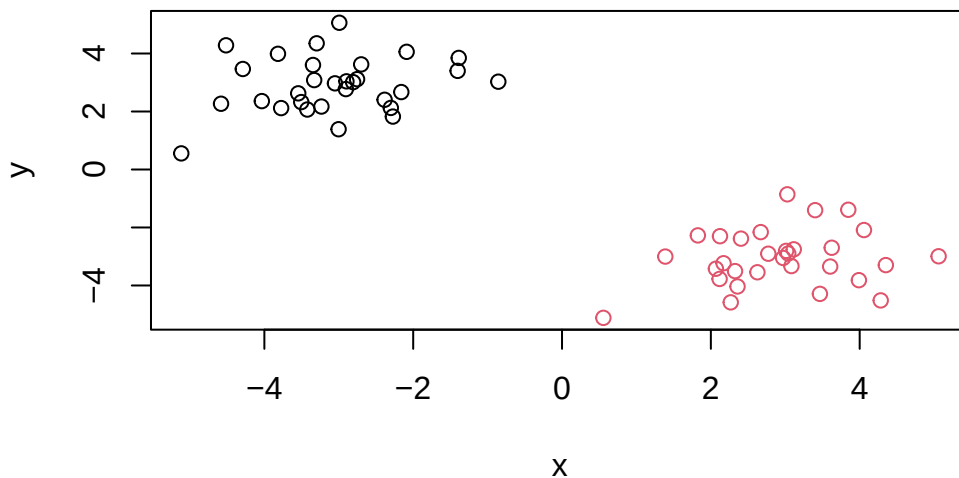
Once we have our `hclust` object (our “tree” of cluster dendrogram”) we can “*cut*” the tree to reveal the clustering pattern.

```
cutree(hc, h=8)
```

[illegible]

Q. Make a plot of $\mathbf{z}$ with your hclust results (i.e. colored by cluster membership)

```
groups <- cutree(hc, k = 2)
plot(z, col = groups)
```



Principal Component Analysis (PCA)

PCA is a dimensionality reduction method that is popular for revealing patterns in complex data sets. PCA takes multi-dimensional datasets and project them into useful ways, such as making figures. PCA aims to reduce dimensionality in an effective way, not all columns are necessary to evaluate the data. In the most simplistic model, with just x and y dimensions, PCA fits a line between data, such as a regression.

Analysis of UK Food Data

Lets look at some data on the eating habbits of people from the UK and evalaute them to see if any patterns or trends that have some regions being distinct from others.

##Data Import

the dad is made available in CSV formate so we can use `read.csv()` to import into R:

```
url <- "https://tinyurl.com/UK-foods"
x <- read.csv(url)
x
```

		X	England	Wales	Scotland	N.Ireland
1	Cheese		105	103	103	66
2	Carcass_meat		245	227	242	267
3	Other_meat		685	803	750	586
4	Fish		147	160	122	93
5	Fats_and_oils		193	235	184	209
6	Sugars		156	175	147	139
7	Fresh_potatoes		720	874	566	1033
8	Fresh_Veg		253	265	171	143
9	Other_Veg		488	570	418	355
10	Processed_potatoes		198	203	220	187
11	Processed_Veg		360	365	337	334
12	Fresh_fruit		1102	1137	957	674
13	Cereals		1472	1582	1462	1494
14	Beverages		57	73	53	47
15	Soft_drinks		1374	1256	1572	1506
16	Alcoholic_drinks		375	475	458	135
17	Confectionery		54	64	62	41

Q1. How many rows and columns are in your new data frame named x? What R functions could you use to answer this questions?

```
nrow(x)
```

```
[1] 17
```

Tidy Data

Fix anything that went wrong with data import.

Previewing the First 6 rows of the data

```
head(x)
```

	X	England	Wales	Scotland	N.Ireland
1	Cheese	105	103	103	66
2	Carcass_meat	245	227	242	267
3	Other_meat	685	803	750	586
4	Fish	147	160	122	93
5	Fats_and_oils	193	235	184	209
6	Sugars	156	175	147	139

```
dim(x)
```

```
[1] 17 5
```

```
rownames(x) <- x[,1]  
x <- x[,-1]  
head(x)
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139

```
dim(x)
```

```
[1] 17 4
```

It is possible to set the row.names when reading the CSV file in, such as the code below:

```
x <- read.csv(url, row.names=1)  
head(x)
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139

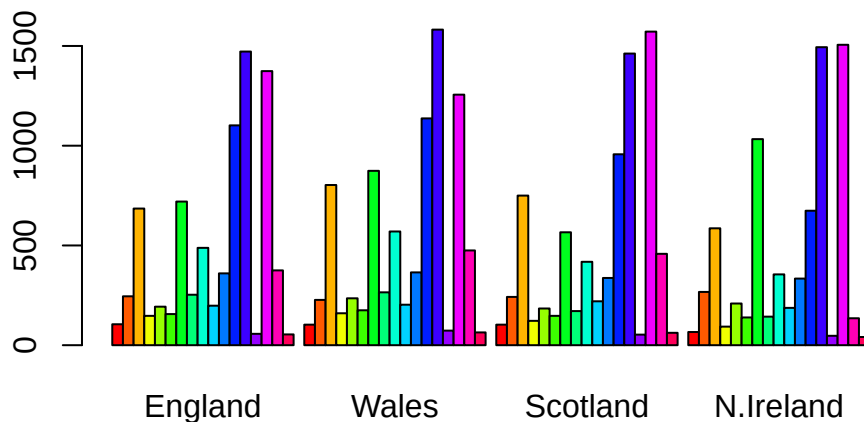
Q2. Which approach to solving the ‘row-names problem’ mentioned above do you prefer and why? Is one approach more robust than another under certain circumstances?

I prefer the second approach as it allows the rows to be properly assigned at the start of data analysis when the CSV files are loaded. In addition, if you run the first example of code multiple times, it will continue to remove rows and labels, which could be detrimental to your code.

Exploratory Analysis

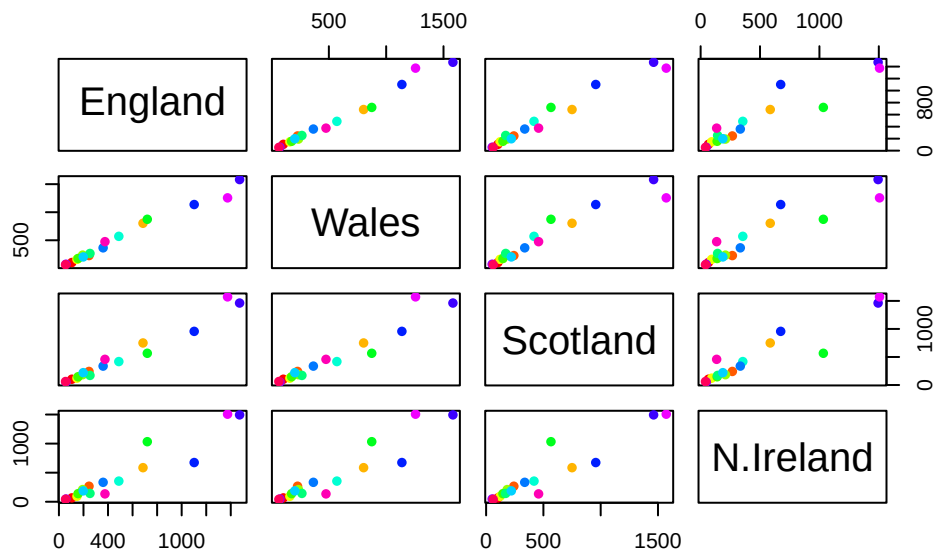
Make some plots to help make sense of obvious trends ...

```
barplot(as.matrix(x), beside=T, col=rainbow(nrow(x)))
```



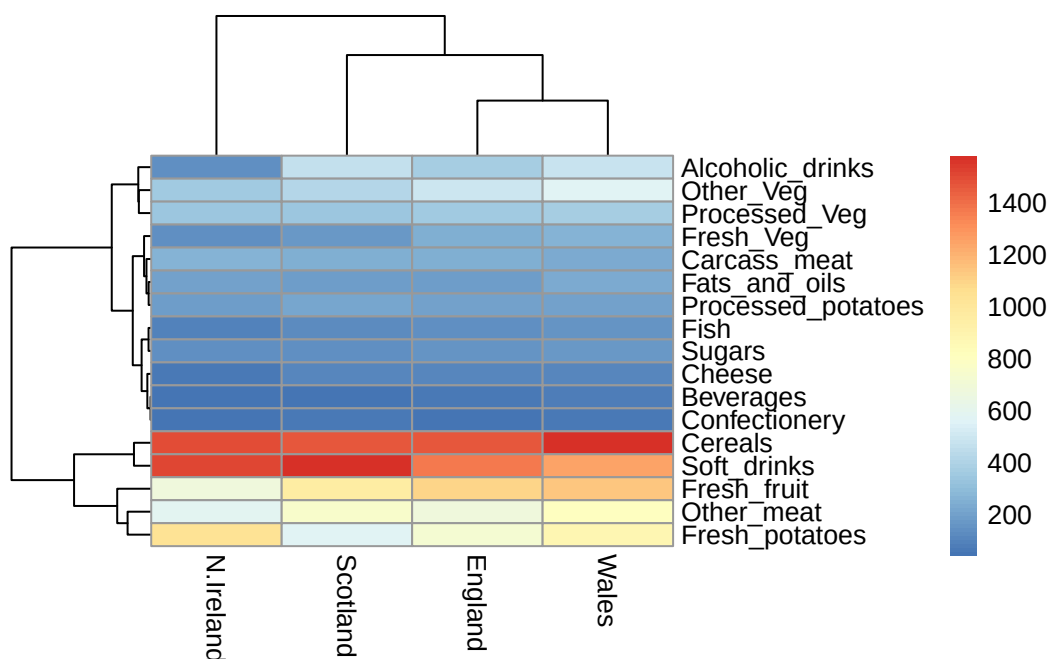
Q5: We can use the `pairs()` function to generate all pairwise plots for our countries. Can you make sense of the following code and resulting figure? What does it mean if a given point lies on the diagonal for a given plot?

```
pairs(x, col=rainbow(nrow(x)), pch=16)
```



```
library(pheatmap)

pheatmap( as.matrix(x) )
```



Key-point: Even relatively small data sets can prove challenging to interpret. Given that it is quite difficult to make sense of even this relatively small data set. Hopefully, we can clearly see that a powerful analytical method is absolutely necessary if we wish to observe trends and patterns in larger data sets.

PCA

The main function in “base” R for PCA is called `prcomp()`. This function wants the “observations” to be rows and the “variables” to be columns.

This block of code will transpose “x” input objects

```
pca <- prcomp( t(x) )
summary(pca)
```

Importance of components:

	PC1	PC2	PC3	PC4
Standard deviation	324.1502	212.7478	73.87622	2.7e-14
Proportion of Variance	0.6744	0.2905	0.03503	0.0e+00
Cumulative Proportion	0.6744	0.9650	1.00000	1.0e+00

The returned PCA object has components that we can use to make our main result figures:

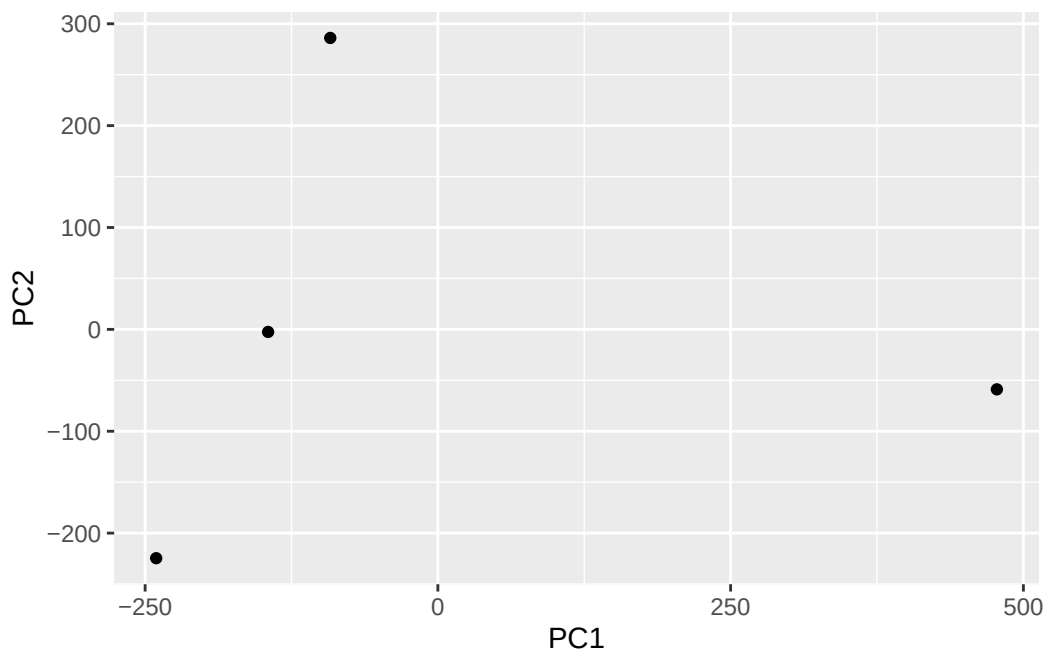
```
attributes(pca)
```

```
$names  
[1] "sdev"      "rotation" "center"    "scale"     "x"  
  
$class  
[1] "prcomp"
```

The main result figure from this analysis is called a “PC Score plot” or “ordination plot” or “PC Plot” or PC1 vs PC2 plot. How does your data lie along the new things that we have calculated? This is a reduced dimension space, reducing what is needed to describe the relationship between these countries, for example.

This plot shows how samples (in this case countries) relate to each other along our new PC axis. (PC1 carries 67% of the variance, so it is more important as a representation of the data.)

```
library(ggplot2)  
  
ggplot(pca$x) +  
  aes(PC1, PC2) +  
  geom_point()
```

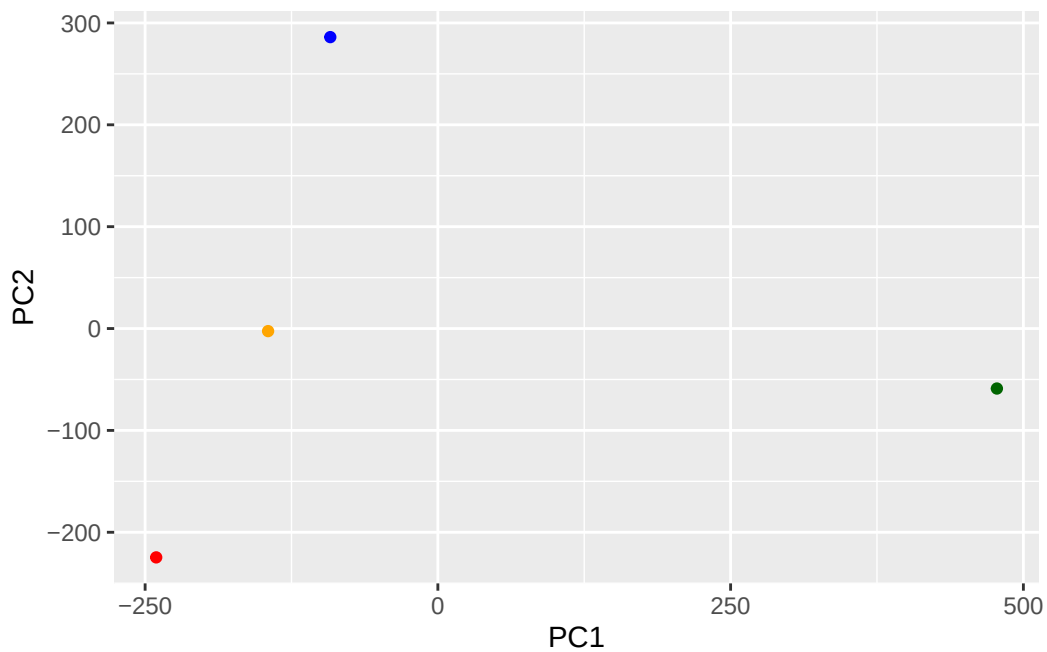



```
countrycols <- c("orange", "red", "blue", "darkgreen")
```

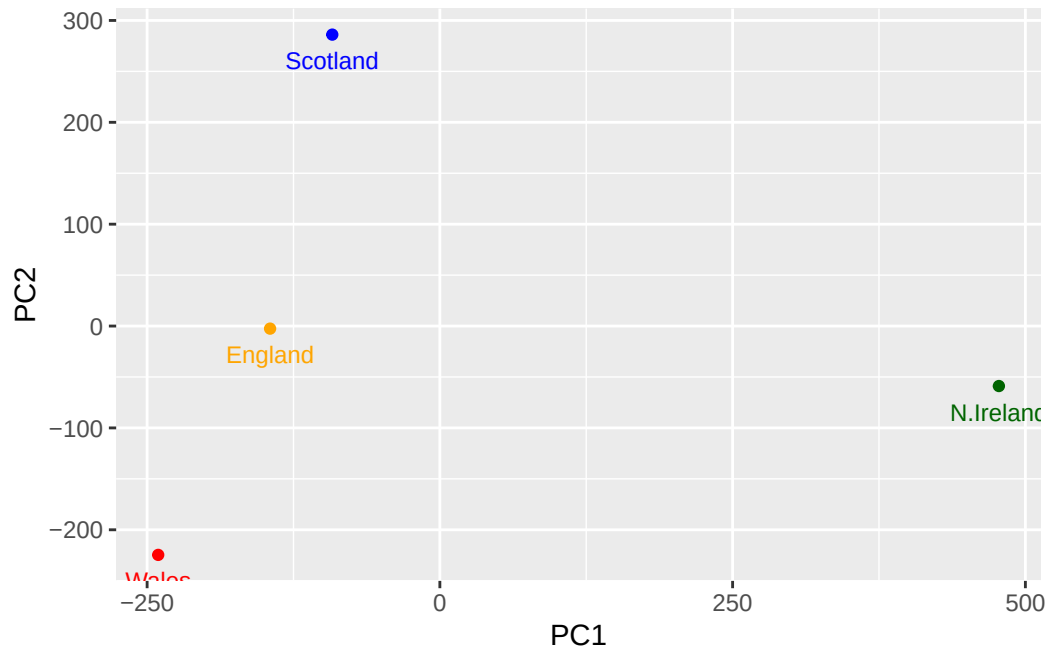
```
pca$x
```

	PC1	PC2	PC3	PC4
England	-144.99315	-2.532999	105.768945	1.612425e-14
Wales	-240.52915	-224.646925	-56.475555	4.751043e-13
Scotland	-91.86934	286.081786	-44.415495	-6.044349e-13
N.Ireland	477.39164	-58.901862	-4.877895	1.145386e-13

```
ggplot(pca$x) +  
  aes(PC1, PC2, label) +  
  geom_point( col = countrycols )
```

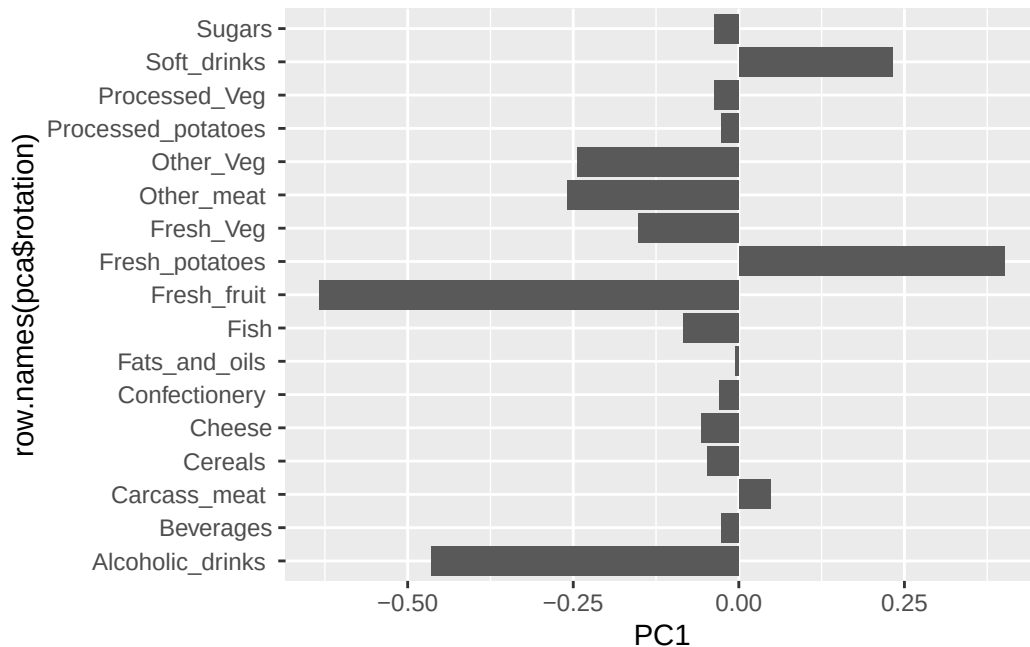


```
ggplot(pca$x) +  
  aes(PC1, PC2, label = row.names(pca$x) ) +  
  geom_point( col = countrycols ) +  
  geom_text(size = 3, vjust=2, col=countrycols)
```



We can see that Northern Ireland is the most different as calculated by PCA, but why? Using `pca$rotation` we can observe all the calculations for each PC as calculated by PCA.

```
ggplot(pca$rotation) +  
  aes(PC1, row.names(pca$rotation)) +  
  geom_col()
```



The barchart of columns above shows that fresh potatoes and soft drinks contribute heavily to N. Irelands skewed results and high PCA variance.

IMPORTANT TAKEAWAYS PC Score Plots, Loading Plots, and Scree plots are the most important takeaways from today’s lab. A PC score plot shows how observations score within each principal component, with the axes usually representing the 2 most important components that help to clarify and explain trends in the data. A loading plot highlights all of the variables that are weighted by principal components and show what variables contribute heavily to the PC score. A scree plot describes variance, ultimately displaying each principal component and what the level of variance each new principal component will highlight. Most informaticians will use a Scree Plot to find the “elbow” flexion point to analyze which principal components are truly needed to fully analyze and interpret the data.